

Credit Card Approval Prediction System

Project Description:

Credit Card Approval Prediction System is a Machine Learning-based web application designed to predict whether a credit card application is likely to be approved based on an applicant's personal, financial, and employment details. The system helps applicants estimate their approval chances before applying, reducing uncertainty and assisting financial institutions in making faster preliminary decisions.

The application allows users to enter details such as gender, income, education, employment status, family status, housing type, occupation, age, and other relevant information. Using a trained Scikit-learn Machine Learning model, the system analyzes these inputs and instantly predicts whether the credit card application is likely to be Approved or Rejected.

Built using Python, Flask, HTML, CSS, and Scikit-learn, the application provides a simple, responsive, and user-friendly interface. The trained Machine Learning model is integrated with the Flask backend to deliver fast and accurate predictions. The project follows clean coding practices, is managed using GitHub, and is deployed on Render for easy online access.

Smart Lender is a Machine Learning-based web application designed to predict whether a loan application is likely to be approved based on the applicant's financial and personal details. The system helps users estimate their loan eligibility before submitting an application, reducing uncertainty and saving time for both applicants and financial institutions.

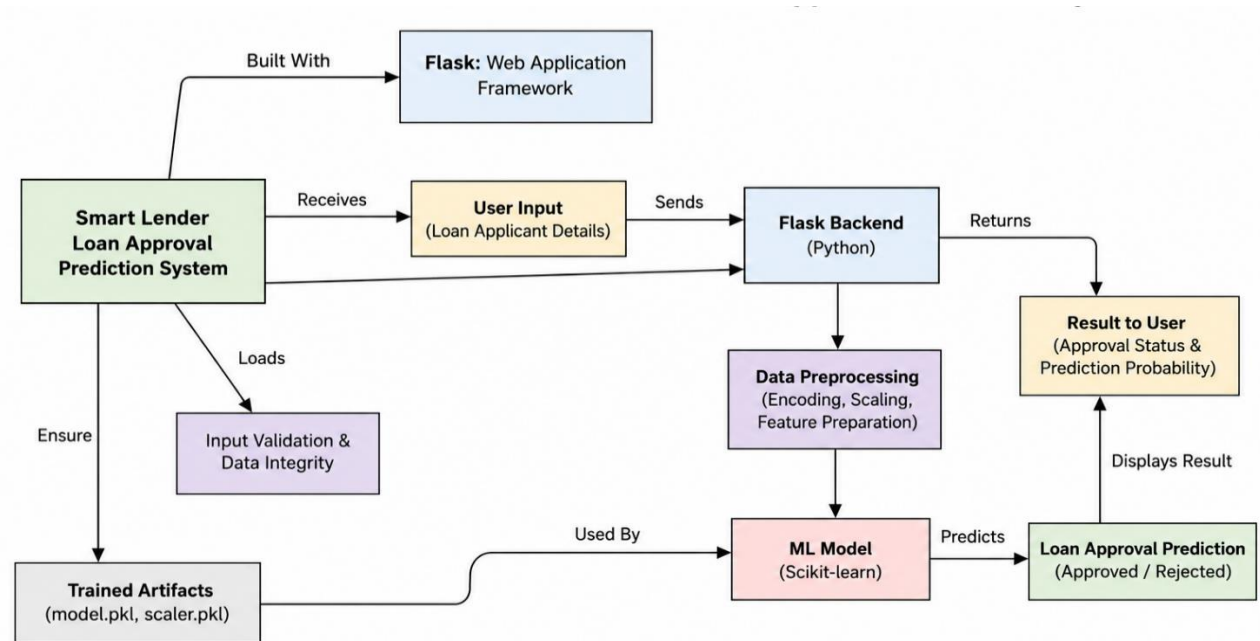
Scenarios

Scenario 1: An individual wants to apply for a new credit card. After entering personal and financial details into the application, the system predicts whether the credit card application is likely to be approved, helping the applicant make an informed decision.

Scenario 2: A working professional checks their credit card eligibility before submitting an application. The system evaluates details such as income, employment status, education, and family information to provide an instant approval prediction.

Scenario 3: A bank employee uses the Credit Card Approval Prediction System to perform an initial assessment of customer applications. The system quickly predicts the approval outcome, reducing manual effort and speeding up the screening process.

Technical Architecture:



Description:

The Credit Card Approval Prediction System follows a three-tier architecture consisting of a Presentation Layer, Application Layer, and Machine Learning Layer. The frontend is developed using HTML and CSS to collect applicant details, while the Flask (Python) backend processes the input data, performs label encoding, and communicates with the trained Random Forest Machine Learning model. Using the stored model.pkl and label_encoders.pkl files, the system predicts whether a credit card application is likely to be Approved or Rejected and displays the result through a simple, responsive web.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Scikit-learn Knowledge:** [Scikit-learn Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Pandas & NumPy Knowledge:** [Pandas and NumPy Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Visual Studio Code Documentation](#)

Project Workflow:

Milestone 1: Data Collection and Model Development

- **Activity 1.1:** Collect and analyze the credit card approval dataset containing applicant info
- **Activity 1.2:** Perform data preprocessing, including handling missing values, encoding categorical features, and preparing the dataset for training.
- **Activity 1.3:** Train and evaluate the Machine Learning model using Scikit-learn and select the best-performing Random Forest model.
- **Activity 1.4:** Save the trained model (model.pkl) and label encoders (label_encoders.pkl) for integration with the web application.

Milestone 2: Backend Development

- **Activity 2.1:** Develop the Flask backend to handle user requests, routing, and prediction processing.
- **Activity 2.2:** Integrate the trained Machine Learning model with the Flask application to generate real-time credit card approval predictions.

Milestone 3: Application Development

- **Activity 3.1:** Develop the main application logic in **app.py**, create Flask routes, and integrate the trained Machine Learning model.
- **Activity 3.2:** Implement input validation, label encoding, and feature preparation before sending data to the prediction model.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop responsive web pages using **HTML** and **CSS** for user interaction.
- **Activity 4.2:** Create Flask templates to collect applicant information and display the credit card approval prediction with confidence score.

Milestone 5: Deployment

- **Activity 5.1:** Configure the project environment and install all required Python dependencies.
- **Activity 5.2:** Deploy the Credit Card Approval Prediction System on **Render** and verify that the application functions correctly in the hosted environment.

Milestone 6: Conclusion

The Credit Card Approval Prediction System successfully integrates Machine Learning with Flask to provide fast and reliable credit card approval predictions. The application offers a simple and user-friendly interface, real-time prediction capability, and a scalable architecture.

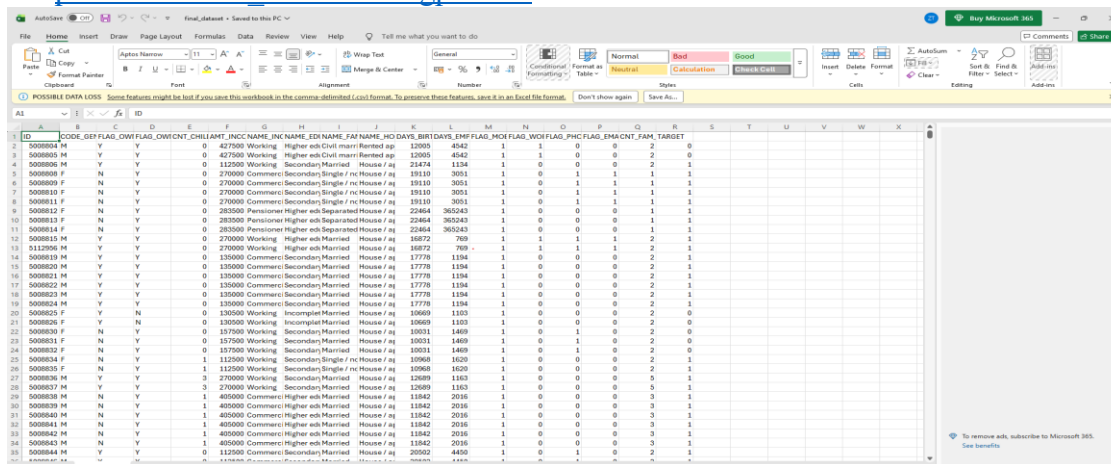
Milestone 1: Data Preparation and Model Development

In this milestone, the focus is on preparing the Credit Card Approval dataset and developing a Machine Learning model capable of predicting credit card approval. This includes collecting the dataset, preprocessing the data, encoding categorical features, training and evaluating different Machine Learning models, and preparing the best-performing model for integration with the Flask web application.

Activity 1.1: Collect and Prepare the Dataset

Before developing the prediction model, a suitable Credit Card Approval dataset is required. Follow the steps below to prepare the dataset for model training.

- **Step 1 — Obtain the Dataset:** Download the Credit Card Approval Prediction Dataset from the Kaggle repository. [data set link](https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction?utm_source=chatgpt.com)
- https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction?utm_source=chatgpt.com.



ID	CODE	GENDER	OCCUPATION	FAMILIARITY	TARGET
0	5008804	M	Working	Higher edu Civil marri	2.0
1	5008805	M	Working	Higher edu Civil marri	2.0
2	5008806	M	Working	SecondaryMarried	2.0
3	5008807	M	Working	SecondaryMarried	2.0
4	5008808	F	Working	SecondaryMarried	1.0
5	5008809	F	Working	SecondaryMarried	1.0
6	5008810	F	Working	SecondaryMarried	1.0
7	5008811	F	Working	SecondaryMarried	1.0
8	5008812	F	Working	SecondaryMarried	1.0
9	5008813	F	Working	SecondaryMarried	1.0
10	5008814	F	Working	SecondaryMarried	1.0
11	5008815	F	Working	SecondaryMarried	1.0
12	5008816	F	Working	SecondaryMarried	1.0
13	5008817	F	Working	SecondaryMarried	1.0
14	5008818	F	Working	SecondaryMarried	1.0
15	5008819	F	Working	SecondaryMarried	1.0
16	5008820	F	Working	SecondaryMarried	1.0
17	5008821	F	Working	SecondaryMarried	1.0
18	5008822	F	Working	SecondaryMarried	1.0
19	5008823	F	Working	SecondaryMarried	1.0
20	5008824	F	Working	SecondaryMarried	1.0
21	5008825	F	Working	SecondaryMarried	1.0
22	5008826	F	Working	SecondaryMarried	1.0
23	5008827	F	Working	SecondaryMarried	1.0
24	5008828	F	Working	SecondaryMarried	1.0
25	5008829	F	Working	SecondaryMarried	1.0
26	5008830	F	Working	SecondaryMarried	1.0
27	5008831	F	Working	SecondaryMarried	1.0
28	5008832	F	Working	SecondaryMarried	1.0
29	5008833	F	Working	SecondaryMarried	1.0
30	5008834	F	Working	SecondaryMarried	1.0
31	5008835	F	Working	SecondaryMarried	1.0
32	5008836	F	Working	SecondaryMarried	1.0
33	5008837	F	Working	SecondaryMarried	1.0
34	5008838	F	Working	SecondaryMarried	1.0
35	5008839	F	Working	SecondaryMarried	1.0
36	5008840	F	Working	SecondaryMarried	1.0
37	5008841	F	Working	SecondaryMarried	1.0
38	5008842	F	Working	SecondaryMarried	1.0
39	5008843	F	Working	SecondaryMarried	1.0
40	5008844	F	Working	SecondaryMarried	1.0

- **Step 2 —Load the Dataset:** Import the application_record.csv and credit_record.csv files into the project using the Pandas library.

```
src > data_analysis.py > ...
15      confusion_matrix,
16      classification_report
17  )
18
19  import joblib
20
21  # =====
22  # Read the datasets
23  # =====
24  app = pd.read_csv("dataset/application_record.csv")
25  ...
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
09-07-2026 12:30 1,523 feature_encoding.py
09-07-2026 13:04 4,116 model_training.py
4 File(s) 11,470 bytes
2 Dir(s) 17,266,913,280 bytes free

C:\Users\zuhat\OneDrive\Desktop\Credit_Card_Approval_Prediction\src>cd ..
C:\Users\zuhat\OneDrive\Desktop\Credit_Card_Approval_Prediction>python src\data_analysis.py
Application Dataset
ID CODE_GENDER ... OCCUPATION_TYPE CNT_FAM_MEMBERS
0 5008804 M ... NaN 2.0
1 5008805 M ... NaN 2.0
2 5008806 M ... Security staff 2.0
3 5008808 F ... Sales staff 1.0
4 5008809 F ... Sales staff 1.0
[5 rows x 18 columns]
```

- **Step 3 — Explore the Dataset:** Analyze the dataset by checking column names, missing values, data types, and statistical summaries.

```

src > data_analysis.py
47 # Dataset information
48 print("\nApplication Dataset Info:")
49 app.info()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Application Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 438557 entries, 0 to 438556
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                     438557 non-null  int64
1   CODE_GENDER                           438557 non-null  str
2   FLAG_OWN_CAR                           438557 non-null  str
3   FLAG_OWN_REALTY                        438557 non-null  str
4   CNT_CHILDREN                           438557 non-null  int64
5   AMT_INCOME_TOTAL                       438557 non-null  float64
6   NAME_INCOME_TYPE                       438557 non-null  str
7   NAME_EDUCATION_TYPE                    438557 non-null  str
8   NAME_FAMILY_STATUS                     438557 non-null  str
9   NAME_HOUSING_TYPE                      438557 non-null  str
10  DAYS_BIRTH                             438557 non-null  int64
11  DAYS_EMPLOYED                           438557 non-null  int64
12  FLAG_MOBIL                             438557 non-null  int64
13  FLAG_WORK_PHONE                         438557 non-null  int64
14  FLAG_PHONE                             438557 non-null  int64

```

- **Step 4 — Clean the Dataset:** Merge the application and credit datasets, create the target variable, and save the processed dataset for feature encoding and model training.

```

63 print("\nMissing Values in Application Dataset:")
64 print(app.isnull().sum())
65

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Missing Values in Application Dataset:
ID                                     0
CODE_GENDER                           0
FLAG_OWN_CAR                           0
FLAG_OWN_REALTY                        0
CNT_CHILDREN                           0
AMT_INCOME_TOTAL                       0
NAME_INCOME_TYPE                       0
NAME_EDUCATION_TYPE                    0
NAME_FAMILY_STATUS                     0
NAME_HOUSING_TYPE                      0
DAYS_BIRTH                             0

```

- **Step 5 — Encode Categorical Data:** Convert categorical features such as gender, income type, education, family status, housing type, and vehicle ownership into numerical values using Label Encoding.
- **Step 6 — Prepare the Training Data:** Separate the processed dataset into input features (**X**) and the target variable (**TARGET**), then split the data into training and testing sets for Machine Learning model development.
- **Important** — Save the trained Random Forest model (model.pkl) and label encoders (label_encoders.pkl) in the models/ folder. These files are loaded by the Flask application to generate real-time credit card approval predictions.

Activity 1.2: Select and Train the Machine Learning Model

The following steps were carried out to select the most suitable Machine Learning model for credit card approval prediction.

- **Understand the Project Requirements:** Review the project objectives and identify the applicant details required to accurately predict whether a credit card application will be approved or rejected.
- **Explore Scikit-learn Documentation:** Study the Machine Learning algorithms, preprocessing techniques, and model evaluation methods available in Scikit-learn for classification tasks.
- **Evaluate Model Performance:** Compare different Machine Learning algorithms, including Logistic Regression, Decision Tree, and Random Forest, based on accuracy and classification performance.
- **Select the Optimal Model:** Choose the Random Forest algorithm as the best-performing model. Train the model using the preprocessed dataset and save the trained model (model.pkl) and label encoders (label_encoders.pkl) for integration with the Flask application.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the Credit Card Approval Prediction System, including the frontend (HTML, CSS), Flask backend, Machine Learning model, and prediction result.
- **Detail Frontend Functionality:** Explain how users enter applicant details such as gender, income, education, employment status, family status, housing type, age, and other information through the web interface.
- **Outline Backend Responsibilities:** Describe how the Flask backend receives user input, validates the data, performs label encoding, prepares the feature vector, and sends it to the trained Machine Learning model.
- **Describe Machine Learning Integration:** Explain how the Flask application loads the trained model.pkl and label_encoders.pkl files, processes the applicant information, predicts whether the credit card application is Approved or Rejected, and displays the prediction result along with the confidence score on the web page.

Activity 1.4: Set Up the Development Environment

- **Install Python and pip:** Ensure Python 3.8 is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>myenv\Scripts\activate  
(myenv) D:\projects> pip install -r requirements.txt
```

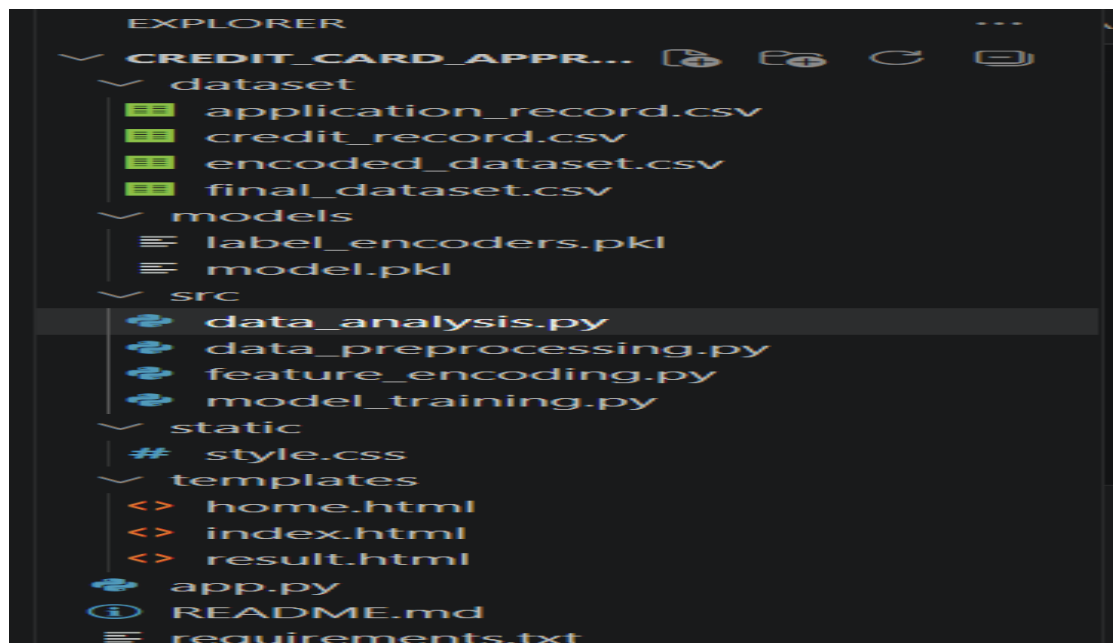
- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask
```

- **Install Required Libraries:** Install the required libraries for Machine Learning and data processing:

```
(myenv) D:\projects> pip install pandas numpy scikit-learn imbalanced-learn joblib
```

- **Configure the Project:** Ensure all the required project files, including the datasets (application_record.csv and credit_record.csv), trained model (model.pkl), and label encoders (label_encoders.pkl), are placed in their respective folders before running the application.
- **Set Up Application Structure:** Create the project directory structure, including folders for dataset, models, templates, static, and src, along with the main application files (app.py, data_analysis.py, data_preprocessing.py, feature_encoding.py, model_training.py, and requirements.txt).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Credit Card Approval Prediction Features

Implement the core functionalities required for the Credit Card Approval Prediction System to provide accurate approval predictions through a user-friendly web interface.

- **Credit Card Approval Prediction:** Predict whether a credit card application is likely to be Approved or Rejected based on the applicant's details using the trained Random Forest Machine Learning model.
- **Data Preprocessing:** Process user inputs by applying the same preprocessing techniques used during model training, including Label Encoding for categorical features before making predictions.
- **Prediction Result:** Display the credit card approval prediction along with the **confidence score** instantly through the Flask web application.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in `app.py` for the **Home Page**, **Prediction Page**, and **Prediction Result**.

Process User Input:

- Create a form in `index.html` for users to enter applicant details such as gender, income, education, employment status, family status, housing type, number of children, age, employment duration, mobile, phone, email, and family members.
- Use Flask's `request.form` to retrieve the submitted user data from the frontend..
- Validate the input values and encode categorical features before processing the prediction request.

Integrate The Machine Learning Model:

- Load the trained Random Forest model (`model.pkl`) and Label Encoders (`label_encoders.pkl`) into the Flask application..
- Apply the same Label Encoding used during model training to the user input.
- Generate the credit card approval prediction and confidence score using the trained model and display the result on the frontend.

Milestone 3: App.py Development

Milestone 3 focuses on developing the app.py file, which serves as the core of the Credit Card Approval Prediction System. In this milestone, Flask routes are created to handle user requests, process applicant details, integrate the trained Machine Learning model, and display the credit card approval prediction through the web application.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for the application's three endpoints, each serving a specific purpose.
 - / — Home page route that renders the main **home.html** interface.

```
@app.route("/")
def home():
    return render_template("home.html")
```

- **/predict**— Displays the applicant input form.

```
@app.route("/predict")
def predict():
    return render_template("index.html")
```

- **/result**— Accepts applicant details submitted through the HTML form, performs label encoding, invokes the trained Random Forest model, and returns the credit card approval.

```
@app.route("/result", methods=["POST"])
def result():

    gender = encoders["CODE_GENDER"].transform(
        [request.form["CODE_GENDER"]]
    )[0]

    own_car = encoders["FLAG_OWN_CAR"].transform(
        [request.form["FLAG_OWN_CAR"]]
    )[0]

    own_realty = encoders["FLAG_OWN_REALTY"].transform(
        [request.form["FLAG_OWN_REALTY"]]
    )[0]

    income_type = encoders["NAME_INCOME_TYPE"].transform(
        [request.form["NAME_INCOME_TYPE"]]
    )[0]
```

```
education = encoders["NAME_EDUCATION_TYPE"].transform(
    [request.form["NAME_EDUCATION_TYPE"]]
)[0]

family = encoders["NAME_FAMILY_STATUS"].transform(
    [request.form["NAME_FAMILY_STATUS"]]
)[0]

housing = encoders["NAME_HOUSING_TYPE"].transform(
    [request.form["NAME_HOUSING_TYPE"]]
)[0]

children = int(request.form["CNT_CHILDREN"])
income = float(request.form["AMT_INCOME_TOTAL"])
days_birth = int(request.form["DAYS_BIRTH"])
days_employed = int(request.form["DAYS_EMPLOYED"])
mobil = int(request.form["FLAG_MOBIL"])
work_phone = int(request.form["FLAG_WORK_PHONE"])
phone = int(request.form["FLAG_PHONE"])
email = int(request.form["FLAG_EMAIL"])
family_members = float(request.form["CNT_FAM_MEMBERS"])

features = [[
    gender,
    own_car,
    own_realty,
    children,
    income,
    income_type,
    education,
    family,
    housing,
    days_birth,
    days_employed,
    mobil,
    work_phone,
    phone,
    email,
    family_members
]]

prediction = model.predict(features)
probability = model.predict_proba(features)
```

```
confidence = round(max(probability[0]) * 100, 2)

if prediction[0] == 1:
    output = "✅ Credit Card Approved"
else:
    output = "❌ Credit Card Rejected"

return render_template(
    "result.html",
    prediction_text=output,
    confidence=confidence
)
```

Set Up Route Handlers for Each Feature:

- The /result route retrieves applicant details submitted through the HTML form using request.form.
- The application validates and converts numerical values while encoding categorical features using the saved Label Encoders.
- The processed data is passed to the trained Random Forest Machine Learning model to generate a prediction.
- The application displays the Credit Card Approved or Credit Card Rejected result along with the confidence score on the result.html page.

Define Machine Learning Prediction Helpers:

- **Load the Trained Model:** Load the trained Random Forest Machine Learning model and Label Encoders when the Flask application starts.

```
model = joblib.load("models/model.pkl")
encoders = joblib.load("models/label_encoders.pkl")
```

- **Retrieve User Input:** Collect the applicant details submitted through the prediction form using Flask's `request.form`.

```
gender = request.form["CODE_GENDER"]
own_car = request.form["FLAG_OWN_CAR"]
income = float(request.form["AMT_INCOME_TOTAL"])
education = request.form["NAME_EDUCATION_TYPE"]
```

- **Preprocess Input Data:** Encode categorical features using the saved Label Encoders and arrange all inputs into a feature vector before prediction.

```
features = [[
    gender,
    own_car,
    own_realty,
    children,
    income,
    income_type,
    education,
    family,
    housing,
    days_birth,
    days_employed,
    mobil,
    work_phone,
    phone,
    email,
    family_members
]]
```

- **Generate Loan Prediction:** Pass the prepared feature vector to the trained Machine Learning model and display the prediction result.

```
prediction = model.predict(features)

confidence = round(
```

```
max(model.predict_proba(features)[0]) * 100,  
2  
)  
  
if prediction[0] == 1:  
    result = "Credit Card Approved ☒  
else:  
    result = "Credit Card Rejected ☐  
  
return render_template(  
    "result.html",  
    prediction_text=result,  
    confidence=confidence  
)
```

Integrate the Machine Learning Prediction in Each Function:

- Load the trained Random Forest model (model.pkl) and Label Encoders (label_encoders.pkl) when the Flask application starts.
- Retrieve the applicant details from the prediction form using request.form.
- Encode the categorical features using the saved Label Encoders and organize all inputs into the required feature vector.

```
prediction = model.predict(features)

confidence = round(
    max(model.predict_proba(features)[0]) * 100,
    2
)
```

- Evaluate the prediction result and determine whether the credit card application is Approved or Rejected.

```
• if prediction[0] == 1:
•     result = "Credit Card Approved ☒ "
• else:
•     result = "Credit Card Rejected ☐ "
•
• return render_template(
•     "result.html",
•     prediction_text=result,
•     confidence=confidence
• )
```

Application Routes:

- **Home route:** / → Renders home.html
- **Prediction Route:** /predict → Displays the loan prediction form (index.html)
- **Result Route (POST):** /result → Accepts applicant details, performs label encoding, generates the prediction using the trained Random Forest model, and displays the result with the confidence score in result.html..

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, responsive, and user-friendly interface for the Credit Card Approval Prediction System. The frontend is developed using HTML and CSS to provide users with an intuitive interface for entering applicant details and viewing the credit card approval prediction generated by the Machine Learning model.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop separate HTML pages (home.html, index.html, and result.html) to provide navigation, collect applicant information, and display the prediction result.
- Use semantic HTML elements such as headings, forms, labels, input fields, select menus, buttons, and containers to organize the user interface.
- Create a simple home page introducing the Credit Card Approval Prediction System and provide a button to navigate to the prediction form.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/style.css) to define the application's appearance, including colors, typography, spacing, buttons, forms, and containers.
- Use a responsive layout to ensure the application is easy to use across different screen sizes.
- Apply button hover effects and consistent styling to improve the overall user experience.

Create Separate UI Components:

- **Home Page:** Displays the Credit Card Approval Prediction System title, a brief project description, and a button to start the prediction process.
- **Prediction Form:** Collects applicant details such as gender, car ownership, real estate ownership, annual income, income type, education, family status, housing type, number of children, days of birth, days employed, mobile, work phone, phone, email, and family members using HTML form elements.
- **Prediction Result:** Displays the Machine Learning prediction as Credit Card Approved or Credit Card Rejected, along with the confidence score, and provides buttons to Predict Again or return to the Home page.

Activity 4.2: Handling User Input Through Flask

Handle Form Submission:

- Create an HTML form in index.html that submits applicant details to the Flask backend using the POST method.
- Validate that all required fields are completed before submitting the form.
- Collect applicant information such as gender, annual income, education, family status, housing type, employment details, and contact information through the form.

Display Prediction Results:

- Process the submitted form through the /result route in app.py.
- Encode the categorical features, prepare the feature vector, and generate the prediction using the trained Random Forest model.
- Pass the prediction result and confidence score from the Flask backend to result.html using render_template().
- Display the prediction as Credit Card Approved or Credit Card Rejected along with the confidence score, and provide navigation buttons to Home or Predict Again.

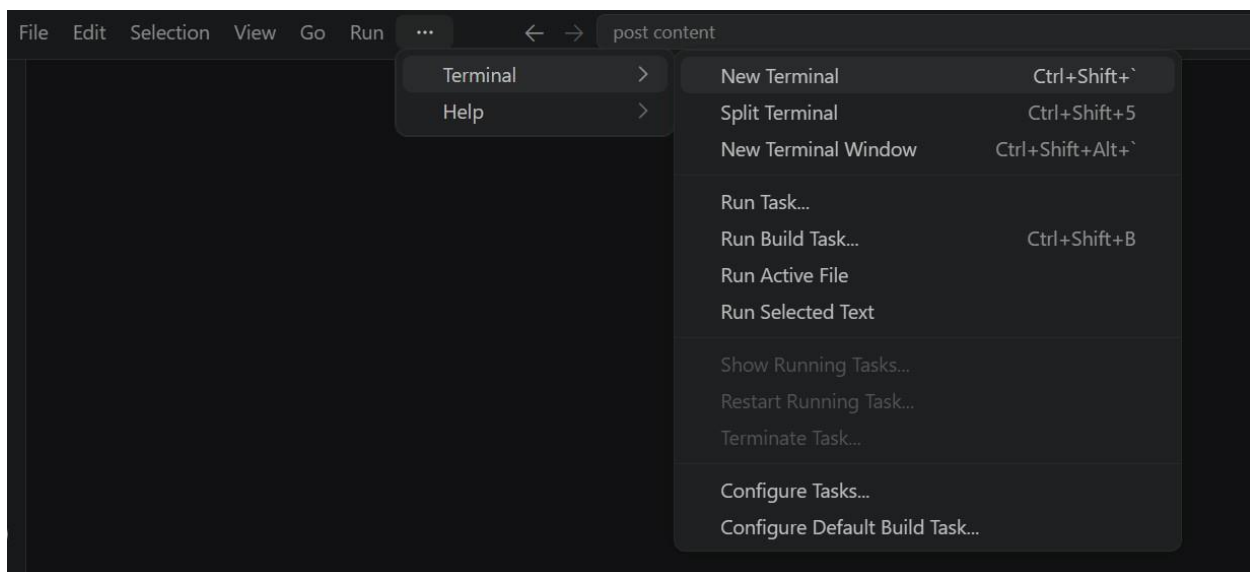
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Credit Card Approval Prediction System locally to verify its functionality and ensure the Machine Learning model integrates correctly with the Flask web application. This includes setting up the development environment, installing the required dependencies, running the Flask server, and testing the prediction system.

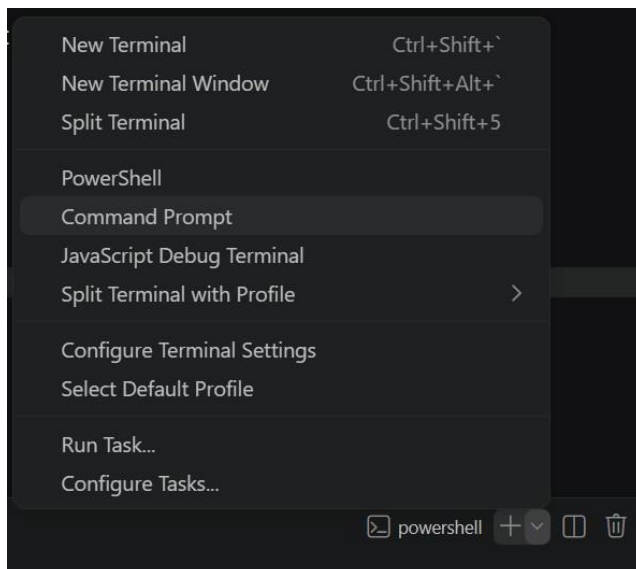
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
C:\Users\u\OneDrive\Desktop\Credit_Card_Approval_Prediction> python -m venv myenv

C:\Users\u\OneDrive\Desktop\Credit_Card_Approval_Prediction>
myenv\Scripts\activate

(myenv) C:\Users\u\OneDrive\Desktop\Credit_Card_Approval_Prediction> pip install -r
requirements.txt
```

Install Required Dependencies:

- Install all the required Python libraries listed in requirements.txt, including Flask, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Joblib, and Unicorn.

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) C:\Users\u\OneDrive\Desktop\Credit_Card_Approval_Prediction> python app.py
```

- Once the server starts successfully, open a web browser and navigate to the <http://127.0.0.1:5000/> to access the application.

```
C:\Users\u\OneDrive\Desktop\Credit_Card_Approval_Prediction>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use
d.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 178-233-492
```

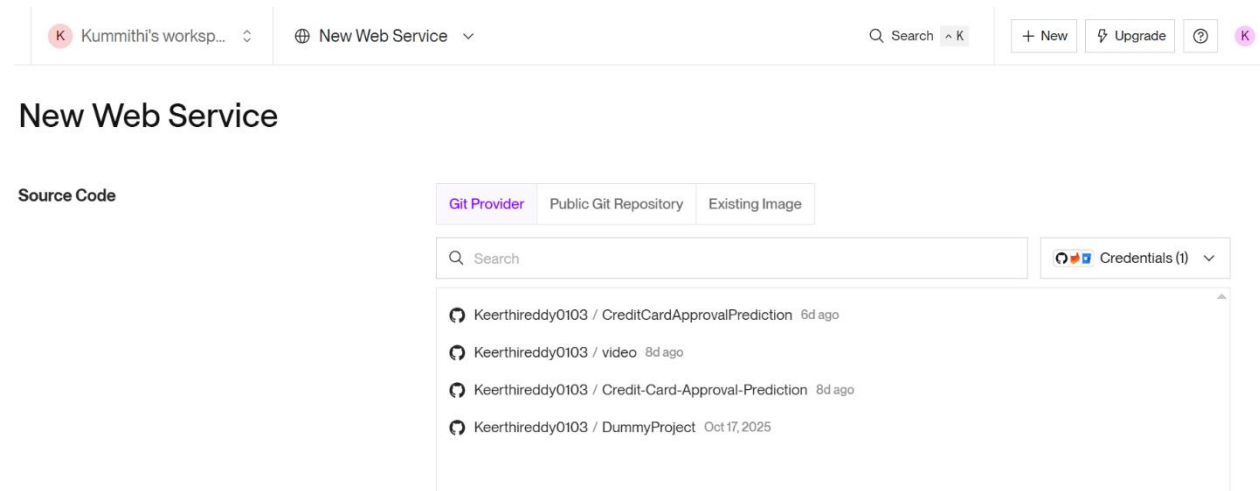
- Interact with the Credit Card Approval Prediction System by entering different applicant details through the prediction form. Verify that the Flask application correctly processes the input, generates the prediction using the trained Random Forest Machine Learning model, and displays the Credit Card Approved or Credit Card Rejected.

Activity 5.3: Public Deployment Using Render

Render is a cloud hosting platform that allows Flask applications to be deployed online, making the Credit Card Approval Prediction System accessible through a secure public URL without requiring users to install the project locally.

Deploy the Application on Render:

- Sign in to the Render dashboard using your GitHub account.
- Click "New +" and select "Web Service".
- Connect your Credit Card GitHub repository to Render.
- Configure the required build and start commands for the Flask application



Configure the Deployment Settings:

- Set the Build Command as:

```
pip install -r requirements.txt
```

- Set the Start Command as:

```
gunicorn app:app
```

- Click Deploy Web Service to start the deployment process.

Access the Public Application:

- After the deployment is completed successfully, Render generates a public URL for the application.

Live Application URL:

```
https://creditcardapprovalprediction-csjq.onrender.com
```

- Open the URL in any web browser to access the **Credit Card Approval Prediction System** from anywhere without installing any software.

=====

🔔 YOUR CREDIT CARD APPROVAL PREDICTION SYSTEM IS LIVE!

=====

Available at your primary URL:

<https://creditcardapprovalprediction-csjq.onrender.com>

=====

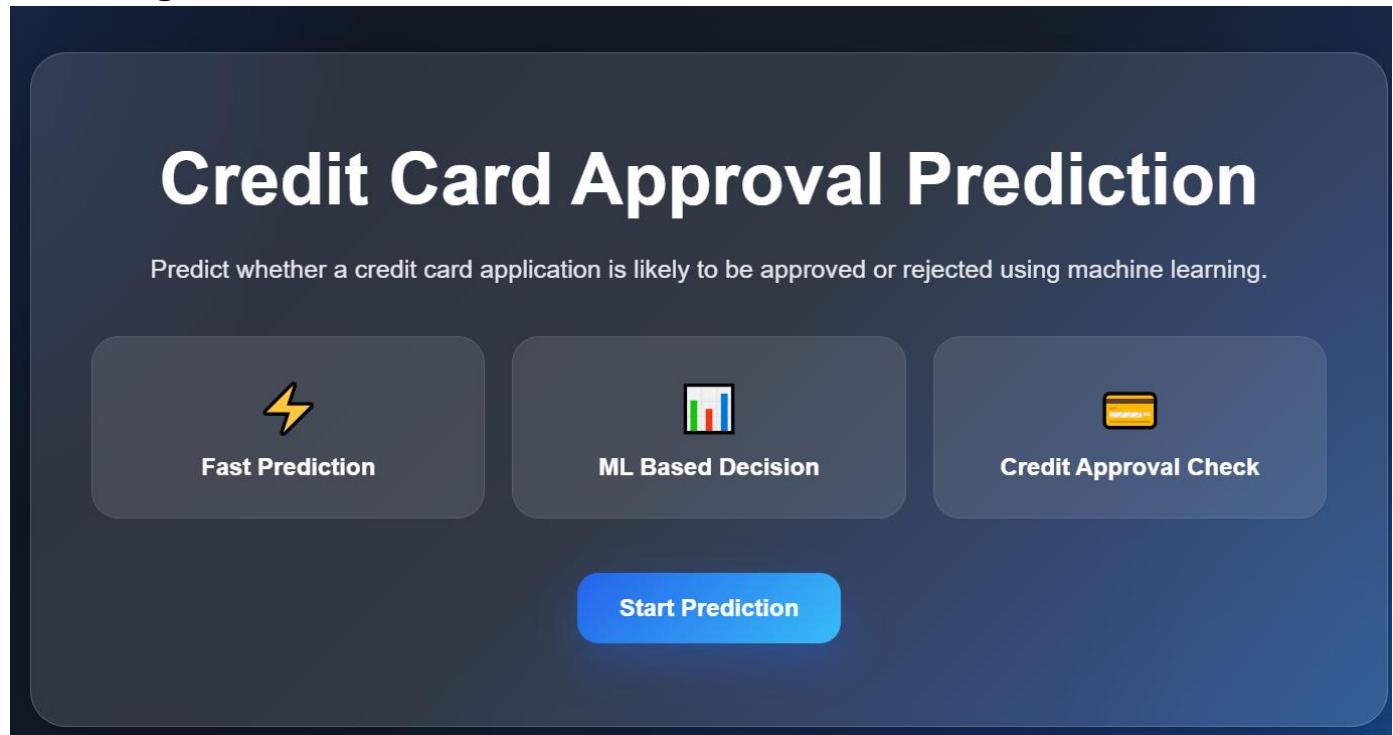
- Share this URL with anyone so they can access the Credit Card Approval Prediction **System** directly from their browser without installing any software.

Important Notes:

- Ensure that all required project files, including the trained Machine Learning model (model.pkl)
- Ensure that all required project files, including the trained Random Forest model (model.pkl) and Label Encoders (label_encoders.pkl), are included before deploying the application.
- The application is deployed on the Render cloud platform and can be accessed using the public deployment URL.
- Whenever changes are pushed to the connected GitHub repository, Render automatically rebuilds and redeploys the application.
- If the application becomes inactive, Render's free plan may take a short time to restart the service when it receives a new request.
- Verify that the application is running successfully by opening the deployed URL and testing different applicant details to ensure the Credit Card Approved or Credit Card Rejected prediction and confidence score are displayed correctly.

Exploring the Website's Web Pages:

Home Page:



Description: The Home Page serves as the landing page of the Credit Card Approval Prediction System. It displays the application title along with a brief description of the system. A "Start Prediction" button is provided, allowing users to navigate to the applicant details form and begin the credit card approval prediction process.

Input Section:

Credit Card Approval Form

Fill in the applicant details to predict approval status.

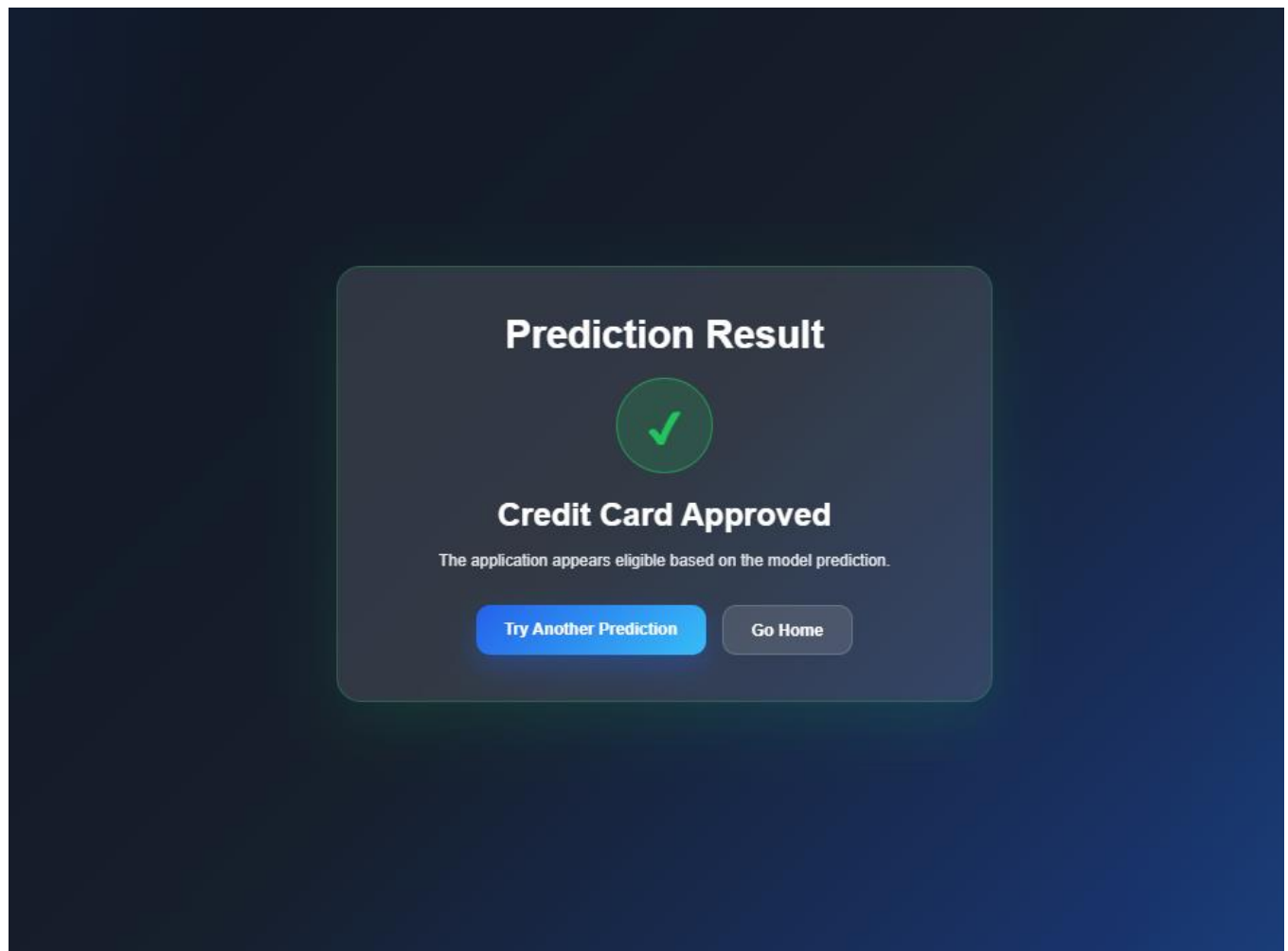
Gender Female ▼	Own Car Yes ▼
Own House / Realty Yes ▼	Number of Children 2 ▼
Total Income 60000 ▼	Income Type Working ▼
Education Type Higher Education ▼	Family Status Married ▼
Housing Type House / Apartment ▼	Days Since Birth -10567 ▼
Days Employed -1030 ▼	Mobile Available Yes ▼
Work Phone No ▼	Phone Yes ▼
Email Yes ▼	Occupation Type IT Staff ▼
Family Members 5 ▼	

Predict Approval

Back

Description: The Input Section provides a user-friendly prediction form where users enter applicant details such as gender, car ownership, real estate ownership, annual income, income type, education, family status, housing type, number of children, age, employment duration, mobile, work phone, phone, email, and family members. After completing the required fields, users click the Predict Approval button. The entered data is sent to the Flask backend, where it is encoded, processed, and passed to the trained Random Forest Machine Learning model to predict whether the credit card application will be Approved or Rejected..

Results Section:



Description: The Prediction Result page displays the prediction generated by the trained Machine Learning model. Based on the applicant's input details, the system displays either "Credit Card Approved" or "Credit Card Rejected" along with the confidence score. The page also provides Home and Predict Again buttons, allowing users to return to the home page or perform another prediction.

Conclusion:

The Credit Card Approval Prediction System is a Machine Learning-based web application developed using Python, Flask, HTML, CSS, and Scikit-learn to predict whether a credit card application is likely to be approved or rejected based on an applicant's personal, financial, and employment details. The system provides users with a fast and reliable prediction, helping them understand their credit card eligibility before submitting an application.

The project involved multiple stages, including data collection, data analysis, preprocessing, feature encoding, model training, model evaluation, frontend and backend development, and deployment. Several Machine Learning algorithms were evaluated, and the Random Forest Classifier was selected due to its superior performance. The trained model was integrated with a Flask web application to provide real-time predictions along with a confidence score through a simple and user-friendly interface.

The application was successfully deployed on the Render cloud platform, making it accessible online without requiring local installation. Version control was managed using GitHub, ensuring proper project management and easy maintenance.

In the future, the system can be enhanced by integrating user authentication, database support for storing applicant information and prediction history, email notifications, interactive dashboards, and more advanced Machine Learning or Deep Learning models to further improve prediction accuracy and user experience. With these enhancements, the Credit Card Approval Prediction System has the potential to become a more comprehensive and intelligent financial decision support application for both applicants and financial institutions.